2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Barabási and Albert (1999) introduced the scale-free network model, demonstrating that real-world complex networks follow power-law degree distributions and that a small number of highly connected hub nodes dominate information flow. This foundational work established the theoretical basis for identifying structurally critical nodes using graph centrality, which our project directly adapts for microservice dependency graphs.

Newman (2010) provided a comprehensive formalization of network centrality metrics, including betweenness centrality—defined as the fraction of all-pairs shortest paths that pass through a given node. This work supplies the mathematical grounding for quantifying which services act as bottlenecks in inter-service communication chains, a role that directly corresponds to cascading failure propagation in distributed systems.

Gan et al. (2019) presented DeathStarBench, a suite of end-to-end microservice benchmarks that faithfully model industry-scale social networks, media streaming platforms, and hotel reservation systems. Their work provides the validated, reproducible experimental substrate on which our fault injection experiments are conducted, ensuring findings are relevant to real-world production deployments.

Sigelman et al. (2010) introduced Dapper-style distributed tracing at Google, establishing the practice of propagating trace context across process boundaries so that the full causal chain of a request can be reconstructed. Jaeger, the open-source implementation adopted in this project, captures parent–child span relationships that enable dynamic construction of service call graphs, overcoming the fundamental limitation of static architecture documentation.

Basiri et al. (2016) formalised Chaos Engineering through Netflix's Chaos Monkey program, establishing the principle of deliberately injecting failures into production-like environments to expose resilience weaknesses proactively. Their methodology of controlled, hypothesis-driven experiments forms the experimental backbone of this project, extended here with graph-theoretic prioritisation of fault targets.

Beyer et al. (2016), in the Google Site Reliability Engineering handbook, articulated service dependency management and blast-radius minimisation as core SRE disciplines. Their framework of error budgets and Service Level Objectives (SLOs) provides the industry motivation for automatically ranking microservices by failure risk—the precise problem our graph centrality approach addresses.

Nedelkoski et al. (2019) investigated fault localisation in microservice architectures using inter-service metric correlation and dependency analysis. While demonstrating the value of service relationships for root cause analysis, their approach required post-failure data. Our work extends this to the proactive dimension: ranking services by predicted blast radius before any incident occurs.

Ma et al. (2020) applied PageRank-inspired walk algorithms to service dependency graphs for anomaly detection, showing that services with high stationary walk probability exhibited greater error propagation rates. This directly motivates the inclusion of PageRank as one of our seven evaluated centrality metrics and aligns with the hypothesis that walk-based importance scores reflect operational failure-propagation likelihood.

2.2 Research Gap

Despite the maturity of graph centrality theory and distributed-tracing infrastructure, a critical research gap persists at their intersection. No prior work has systematically and empirically validated whether static structural centrality metrics—derived from runtime trace-based Service Dependency Graphs (SDGs)—can predict the real-world blast radius of chaos fault-injection events.

Existing fault localisation and anomaly detection literature focuses primarily on reactive root cause analysis: identifying the source of a failure after it has occurred. This project addresses the orthogonal, proactive dimension—can structural graph analysis rank services by failure risk before any incidents occur, enabling pre-emptive resilience investment?

Furthermore, prior art typically employs either (a) static, manually maintained architecture diagrams that become stale in continuously deployed environments, or (b) single-metric evaluations that test only one centrality measure against one failure indicator. This project contributes a seven-metric comparative study evaluated across five independent blast radius dimensions, providing a holistic and statistically rigorous correlation matrix validated against real chaos experiments.

Finally, the systematic identification of outliers—services where centrality predictions diverge significantly from empirical blast radius—has not been studied. Understanding False Negatives (structurally peripheral services that cause disproportionate damage, such as compose-post-service in our study) is essential for defining the limits of pure graph-theoretic risk models and motivating hybrid approaches that combine structural and semantic service knowledge.

2.3 Objectives

The primary objectives of this project are defined below, each formulated as a Specific, Measurable, Achievable, Relevant, and Time-Bound (SMART) goal:

- SDG Extraction: To dynamically construct a directed, weighted Service Dependency Graph from live Jaeger distributed traces collected from the DeathStarBench Social Network application, capturing all inter-service RPC call relationships and their associated latency characteristics.
- Centrality Computation: To compute seven normalised graph centrality metrics—Degree, In-Degree, Out-Degree, Betweenness, Closeness, Eigenvector, and PageRank—for every microservice node in the SDG, using the NetworkX graph analysis library.
- Composite Ranking: To derive a composite centrality score as the mean of normalised per-metric ranks, providing a single aggregate structural importance ranking across all seven dimensions.
- Chaos Fault Injection: To systematically inject two fault types (container kill via SIGKILL and 100 ms network-latency injection via tc netem through Pumba) into each of the 12 application microservices, with two experimental repetitions per (service, fault-type) combination.

• Blast Radius Measurement: To quantitatively measure fault impact across five dimensions—tail latency degradation (p95 and p99), error rate increase, number of downstream services affected, and system recovery time—collected from Jaeger traces during active fault windows.

• Statistical Correlation: To compute Spearman's rank correlation coefficient (ρ) and two-tailed p-values between each centrality metric and each blast radius measure, identifying the graph structural properties that most reliably predict failure impact at the $p < 0.05$ significance level.

• Outlier Analysis: To identify and qualitatively explain services where centrality rank and empirical blast-radius rank diverge by more than N/3 positions, exposing the limitations of static graph metrics and informing future hybrid risk models.

• Visualization: To produce correlation heatmaps, centrality-vs-blast scatter plots, and composite score bar charts enabling communication of findings to non-specialist SRE practitioners.

2.4 Problem Statement

In modern cloud-native architectures composed of dozens of independently deployed microservices, a single service failure can cascade through dependency chains and trigger system-wide outages affecting thousands of concurrent users. Site Reliability Engineers (SREs) currently lack a principled, data-driven methodology to proactively identify which microservices pose the greatest systemic risk, forcing reliance on subjective expertise or reactive post-incident analysis.

This project addresses the following core research problem: Can graph-theoretic centrality metrics, derived from dynamically extracted runtime Service Dependency Graphs, serve as statistically significant and practically useful predictors of the blast radius caused by fault injection in a representative microservice architecture?

The problem encompasses three interrelated sub-challenges: (1) the dynamic, automated construction of accurate SDGs from distributed trace data, avoiding the staleness inherent in manually maintained architecture diagrams; (2) the empirical quantification of blast radius across multiple orthogonal dimensions using controlled chaos experiments; and (3) the rigorous statistical validation of the structural-to-operational centrality correspondence hypothesis.

Successfully solving this problem enables a shift from reactive incident response to proactive resilience engineering, empowering platform teams to quantitatively prioritise investment in redundancy, autoscaling, circuit breakers, and other fault-tolerance mechanisms based on objective graph-structural evidence.

2.5 Project Plan

The project is planned over an eight-month timeline, structured into eight sequential phases aligned with the six-stage experimental pipeline. Each phase produces a concrete deliverable that feeds directly into the subsequent stage. The Gantt chart below illustrates task durations, sequencing, and overlapping phases.

Task	Activity	M1	M2	M3	M4	M5	M6	M7	M8

Task 1	Literature Survey								
Task 2	Environment Setup & Deployment								
Task 3	SDG Extraction Pipeline								
Task 4	Centrality Computation Module								
Task 5	Fault Injection & Measurement								
Task 6	Statistical Analysis & Plots								
Task 7	Testing & Validation								
Task 8	Documentation & Report Writing								

Fig. 1. Project Gantt Chart — Microservice Failure Analysis Pipeline

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

• Trace Collection: The system shall query the Jaeger REST API (/api/traces endpoint) to retrieve distributed trace data from the live DeathStarBench stack within a configurable lookback window and trace-count limit, exporting raw spans as structured JSON.

• SDG Construction: The system shall parse Jaeger trace JSON to resolve parentSpanID→spanID relationships and map processID to service name, constructing a directed weighted NetworkX DiGraph where nodes represent services and edges represent inter-service RPC call dependencies annotated with call count and average latency.

• Graph Export: The system shall export the constructed SDG to both GraphML format (for use with visual tools such as Gephi) and JSON node-link format for downstream programmatic consumption by analysis modules.

• Centrality Computation: The system shall compute seven normalised centrality metrics per service node: Degree, In-Degree, Out-Degree, Betweenness, Closeness, Eigenvector, and PageRank (damping = 0.85), using NetworkX standard library implementations.

• Composite Scoring: The system shall compute a composite centrality score as the mean of per-service normalised ranks across all seven metrics, and produce a composite rank where rank 1 denotes the structurally most critical service.

• Fault Injection Orchestration: The system shall orchestrate per-service chaos cycles using Pumba, supporting Container Kill (SIGKILL) and Network Latency Injection (tc netem) fault types with configurable duration, jitter, and repetition count.

• Blast Radius Measurement: During each fault window, the system shall query Jaeger to compute tail-latency percentiles (p95, p99), error rate fractions, downstream affected service count (services showing >2× baseline latency or new errors), and container recovery time in seconds.

• Statistical Correlation: The system shall compute Spearman's rank correlation coefficient (ρ) and two-tailed p-values for all 40 combinations of eight centrality metrics and five blast radius measures using scipy.stats.spearmanr.

• Visualisation Generation: The system shall produce Spearman correlation heatmaps, per-pair scatter plots annotated with service labels, and composite-score bar charts using Matplotlib and Seaborn.

• Outlier Detection: The system shall identify services flagged as False Positives (high centrality, low blast) or False Negatives (low centrality, high blast) based on absolute rank discrepancy exceeding a configurable fraction threshold (default N/3).

• Integration Testing: After each fault recovery cycle the system shall execute pytest integration tests verifying core user flows (registration, login, post composition, timeline retrieval) to confirm functional system correctness.

• Experiment Logging: The system shall append all experiment run records—service, fault type, repetition, all measured metrics—to a persistent CSV log file for full reproducibility and audit traceability.

3.1.2 Non-Functional

• Performance: Trace collection and SDG construction shall complete within five minutes for a two-hour lookback window with up to 2,000 traces. Centrality computation shall complete within 30 seconds for graphs of up to 100 nodes.

• Scalability: The SDG extraction and centrality computation pipeline shall accommodate microservice applications containing up to 100 service nodes without requiring architectural modification.

• Reliability: The fault injection module shall guarantee container restart and service recovery verification before initiating the next experiment run, preventing state contamination between runs.

• Reproducibility: All stochastic components shall use a fixed random seed (seed = 42). All experiment parameters, trace collection window boundaries, and Pumba command arguments shall be fully logged for exact replication of results.

• Portability: The system shall operate on any platform supporting Docker Desktop 4.0+ and Python 3.11+, including Windows 11 with WSL2 backend, macOS 13+, and Ubuntu 22.04 LTS.

• Maintainability: The codebase shall be organised into independent, single-responsibility Python modules with comprehensive docstrings and type annotations. All modules shall be covered by unit and integration tests using pytest.

• Security: The fault injection module shall operate exclusively within the isolated Docker bridge network. No production systems shall be targeted. The Pumba container shall mount only /var/run/docker.sock.

• Usability: All pipeline phases shall be executable via a single orchestrator script (run_full_experiment.py) with configurable CLI flags. A --dry-run mode shall allow plan inspection without execution.

3.2 Feasibility Study

3.2.1 Technical Feasibility

• Technology Availability: All core technologies—Docker, Python 3.11, Jaeger distributed tracing, NetworkX, SciPy, and Pumba—are mature, open-source, and actively maintained with extensive public documentation and community support.

• Algorithm Availability: All seven centrality metrics are available as single-call standard functions in NetworkX. Spearman's rank correlation is provided by scipy.stats. No novel algorithm development is required; project effort is focused on integration and experimental design.

• Benchmark Availability: DeathStarBench is a peer-reviewed, publicly available benchmark deployable via Docker Compose on commodity hardware, providing a reproducible and representative experimental substrate.

• Hardware Accessibility: The full experiment stack runs on a single developer workstation with 16 GB RAM and 8 CPU cores—equipment accessible in any

university computer science laboratory.

• Integration Feasibility: All components communicate through standard interfaces: HTTP REST (Jaeger API), Docker SDK for Python (container management), and the Linux network namespace (Pumba tc netem injection).

3.2.2 Economic Feasibility

• Zero Licensing Cost: All software components—Python, Docker Community Edition, Jaeger, NetworkX, Pumba, Matplotlib, and pytest—are open-source and available at no cost under permissive licences (Apache 2.0, MIT, BSD).

• Infrastructure Cost: Experiments run on existing university lab hardware or a personal developer laptop. Optional cloud deployment on an AWS EC2 m5.2xlarge spot instance (~$0.10/hour) would cost under ₹500 for a full experiment run.

• Return on Investment: A single prevented enterprise production outage carries an average cost of $300,000 per hour (Gartner, 2022). The methodology produced by this project provides an overwhelming positive ROI by enabling targeted, evidence-based resilience investment.

• Maintenance Cost: The open-source toolchain requires no licence renewals. The Python codebase requires only periodic dependency version updates, estimated at less than one engineering day per quarter.

3.2.3 Social Feasibility

• Industry Relevance: The methodology directly addresses a pain point experienced by Site Reliability Engineering teams at technology organisations of all sizes. The output—a ranked risk table of microservices—is immediately actionable without requiring graph theory expertise.

• User Acceptance: SREs and platform engineers are already familiar with distributed tracing tools (Jaeger, Zipkin) and container orchestration (Docker, Kubernetes). The proposed system builds on existing tooling, minimising adoption friction and training overhead.

• Ethical Considerations: All fault injection experiments are performed in isolated Docker environments on a local developer machine using a benchmark application containing no real user data. No production systems or personal information are involved.

• Impact on Workforce: Automated service risk ranking augments SRE capabilities rather than replacing human engineering judgement. It frees engineers from time-consuming manual dependency audits, enabling focus on higher-level architectural resilience decisions.

• Knowledge Dissemination: Findings will be documented in this project report, and the full experimental pipeline will be made publicly available, enabling the academic community to replicate, extend, and build upon the validated methodology.

3.3 System Specification

3.3.1 Hardware Specification

The following hardware specifications are required to deploy and execute the complete DeathStarBench microservice stack alongside the fault injection orchestrator and

analysis pipeline:

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5 / AMD Ryzen 5 4 cores, ≥ 2.0 GHz	Intel Core i7 / AMD Ryzen 7 8 cores, ≥ 3.0 GHz
Memory (RAM)	8 GB DDR4	16 – 32 GB DDR4
Storage	30 GB free SSD / HDD	50 GB free NVMe SSD
GPU	Not required	Not required (CPU-only)
Network	Local Docker bridge	Gigabit LAN
Monitor	1920 × 1080 (Full HD)	Full HD or higher

Table 1. Hardware Specification

3.3.2 Software Specification

The software ecosystem required to execute the full six-phase experiment pipeline is listed below. All tools are open-source and cross-platform compatible:

Category	Tool / Version	Purpose
Operating System	Windows 11 + WSL2 / Ubuntu 22.04 LTS / macOS 13+	Host environment for Docker & Python
Programming Language	Python 3.11+	All pipeline scripts, analysis & orchestration
Development Environment	VS Code / PyCharm	Code editing, debugging, project management
Graph Library	NetworkX 3.1	SDG construction & 7 centrality metric computation
Data Analysis	Pandas 2.0, NumPy 1.24, SciPy 1.11	Aggregation, ranking, Spearman correlation
Visualization	Matplotlib 3.7, Seaborn 0.12	Heatmaps, scatter plots, bar charts
Container Platform	Docker Desktop 4.0+ (WSL2)	Multi-container microservice deployment
Distributed Tracing	Jaeger All-in-One v1.62.0	Trace collection, span storage, REST API
Load Generation	Locust 2.15 / wrk2 (Docker)	Synthetic HTTP traffic at 50 req/s
Chaos Engineering	Pumba (ghcr.io/alexei-led/pumba)	Container SIGKILL & tc netem latency injection
HTTP Client	Requests 2.31	Jaeger API queries & DeathStarBench API calls
Testing Framework	pytest 7.4	Integration tests for post-fault recovery validation
Build System	setuptools 68.0, pip	Package management & dependency installation

Table 2. Software Specification